

AI 协作反思日志 — Phase 4 核心开发与功能完善

团队：海底小纵队 日期：2026年5月1日

1. 本阶段 AI 使用清单

任务	AI工具	Prompt摘要	AI输出质量 (1-5)	人工修改幅度(%)
需求发布接口与页面	Claude Code	实现 POST /api/v1/requirements + 前端表单页，支持标题/描述/预算/分类校验	4	15%
接单接口	Claude Code	实现 POST /api/v1/orders, 处理 4001 重复接单/4002 自接, @Transactional 保证原子性	4	10%
订单详情与历史列表	Claude Code	实现订单详情查询 + 我接取的/我发布的订单列表, 403 越权保护	3	25%
联网搜索 (项目模板参考)	DeepSeek	查询 Spring Boot + Vue 项目结构模板、MyBatis-Plus 配置示例	4	5%
系统集成联调审查 (5.19)	Claude Code	全面审查前后端 26 个 API 接口一致性、字段匹配、认证传递	5	5% (直接采纳发现结果)
AI 调试对决实验 (5.19)	Claude Code	针对 3 个系统联调 Bug, 提供报错/复现步骤/相关代码, AI 定位并修复	5	0%
AI 代码信任度实验 (5.19)	Claude Code	实现评价提交与信用分计算, 含订单校验、权限、评分映射	3	35% (null 检查、错误码、事务回滚需修复)

2. AI 助力分析 (Q1)

AI 在编码中最有价值的场景：

AI 在生成标准化 CRUD 代码和模板化 API 方面效率极高。本次会话中，AI 一次性生成的三层代码（Controller → Service → DTO）几乎无需修改即可编译通过。尤其在有明确 P3 设计文档约束的情况下，AI 能严格遵循 RESTful 命名规范、统一响应结构和已有错误码体系。

AI 在测试生成中最有价值的场景：

AI 能够自主设计覆盖正常流程和异常边界的测试用例，并直接在终端执行 curl 命令验证。本次会话中 AI 自动完成了 30+ 条 API 测试，覆盖注册、登录、发布、接单、越权、重复操作等场景，且能根据测试结果自主调试和修复。

AI 在调试中最有价值的场景：

AI 能从错误堆栈中快速定位根因。例如"服务端异常"无具体错误信息时，AI 主动在 GlobalExceptionHandler 中添加日志输出，然后从 NumberFormatException: For input string: "publish" 追溯到前端路由匹配问题。

3. AI 误导分析（Q2）

"AI 代码信任度实验"数据摘要（更新于 2026-05-19）

以"评价提交与信用分计算"功能为例，真实替换代码并运行 11 个单元测试：

- **AI 直出的测试通过率：** 4/11 = 36%
- **人工审查修复后的测试通过率：** 11/11 = 100%
- **AI 直出代码的真实表现：**
 - 4 个测试通过（基础信用分计算逻辑正确）
 - 7 个测试失败：
 - 1 个 NPE（requirement null 未检查）
 - 2 个错误码不一致（400 vs 4003, 403 vs 4004）
 - 1 个冗余 DB 写入（scoreChange=0 时仍 update）
 - 3 个通知类型/参数不匹配
- **发现的主要问题类型：**
 - i. 防御性编程缺失（3 个 null 场景全漏）
 - ii. 项目编码规范破损（错误码体系、通知命名约定）
 - iii. 事务配置不完整（@Transactional 缺少 rollbackFor）
 - iv. 无优化意识（3 星评价冗余 SQL）

"AI 调试对决"实验总结

早期会话中遇到和修复的 5 个典型 Bug：

Bug #	Bug 描述	AI 定位耗时	AI 定位是否准确	AI 修复方案是否可用	最终方案来源
1	注册时空字符串 student_id 致 uk_student_id 冲突	<2min	准确	可用	AI
2	GET /requirements/publish 返回 500	<2min	准确	可用	AI
3	Snowflake ID 在浏览器中精度丢失	<2min	准确	可用	AI
4	前端页面始终报 404/500，后端 API 却正常	~15min	逐步定位	需人工判断	AI + 人工

Bug #	Bug 描述	AI 定位耗时	AI 定位是否准确	AI 修复方案是否可用	最终方案来源
5	"我发布的"列表不显示 PENDING 需求	<2min	准确	可用	AI

2026-05-19 系统集成联调新增 3 个 Bug（任务 5）：

Bug #	Bug 描述	AI 定位耗时	AI 定位是否准确	AI 修复方案是否可用	最终方案来源
6	分页 total 被前端 activeRequirements().length 覆盖	<1min	准确	可用（还修了两处）	AI
7	筛选下拉 LOST_FOUND 后端枚举无此值	<1min	准确	可用（附带发现遗漏类型）	AI
8	typeLabel 函数枚举名与后端不一致 (4 处)	<1min	准确	可用（两文件同步修复）	AI

3 个新增 Bug 的纯人工 vs AI 对比：

- 人工定位耗时（估）：10-15 分钟/个
- AI 定位耗时：< 1 分钟/个（提速 10-15 倍）
- AI 定位准确度：3/3 准确（100%）
- AI 修复方案可采纳率：3/3（100%）
- AI 额外价值：Bug #7 附带发现筛选下拉缺少 5 种有效类型

Bug #4 详细过程（最有价值的调试案例）：

- 现象：用户浏览器访问 /requirements/publish 始终报错，但 curl 测试 API 全部通过
- AI 排查路径：① 检查后端日志 → ② 发现 NoResourceFoundException → ③ 怀疑前端路由未生效 → ④ curl 拉取前端 JS 文件验证路由 → ⑤ 发现 index.html 加载 main.js 而非 main.ts → ⑥ 确认 main.js 中缺少 publish 路由
- 耗时约 15 分钟，涉及前后端 + 网络请求的全链路排查

AI 擅长调试的 Bug 类型：

- 堆栈明确的异常（如 DuplicateKeyException、NumberFormatException）
- 配置/路由类问题（路径不匹配、文件加载错误）
- 类型转换/序列化问题（JSON 精度、Long/String）

AI 不擅长调试的 Bug 类型：

- 需要理解用户实际业务流程的逻辑偏差（如"我发布的"应该包含 PENDING 需求）
- 需要多轮用户反馈才能确认的交互问题

AI 修复方案的典型问题：

- 倾向于"修症状"而非"改设计": 如对 GET /publish 的 500, AI 最初只加了 404 handler, 后来才意识到应该从路由层面阻止非数字匹配
- 对"不应该出现的文件"缺乏敏感度: 如 main.js 和 main.ts 双入口问题, AI 一开始直接忽略了 main.js 的存在

4. Prompt 改进计划 (Q3)

编码阶段的 Prompt 最佳实践:

- 提供具体的完成标准 (如"预算小于 0 返回 400"、"重复接单返回 4001"), AI 能据此精确实现
- 引用已有的设计文档 (如 P3 API 规范), AI 能严格遵循既定接口契约
- 要求"自行测试", AI 会自动执行 curl 测试并修复发现的问题

调试阶段的 Prompt 最佳实践:

- 提供完整的错误堆栈和复现步骤, AI 定位效率最高
- 对于跨层问题 (前端→网络→后端→数据库), 提供每层的状态信息 (数据库查询结果、curl 测试结果、浏览器控制台输出)
- 明确要求"记录排查过程到文档", AI 会在排查时更系统地记录步骤

下阶段计划改进:

- 在 AI 生成业务逻辑代码前, 先用自然语言描述完整的业务流程 (含异常分支), 让 AI 确认理解后再编码
- 对于涉及两个入口文件/配置文件的场景, 要求 AI 先列出所有相关文件避免遗漏
- 建立"常见 AI 遗漏清单" (如 JS 精度、空字符串 vs null、前端实际入口文件), 在 Prompt 中预加载

2026-05-19 新增 Prompt 经验 (来自任务 3 和任务 5):

- **前后端一致性检查的 Prompt 模板:** 指定"逐一对比后端 Controller 路径与前端 API 封装函数"可获得极高准确度
- **AI 代码生成的 Prompt 改进:** Prompt 中应明确声明项目编码规范 (错误码体系、通知命名约定、null 处理策略), 否则 AI 会用通用方案替代
- **调试 Prompt 的关键信息:** 提供完整的文件路径列表比描述问题更能提高 AI 定位效率 (本次 3 个 Bug 均在 <1min 内定位)

5. 本阶段的核心工程判断

决策 1: Docker 本地数据库 vs 共享数据库

- 为什么 AI 无法单独做这个决策: AI 不了解团队的协作模式、网络环境、数据安全要求
- 团队最终判断: 采用 docker-compose.yml 本地 MySQL + mysql-init/init.sql 初始化, 数据目录 mysql-data/ 挂载到本地。每位开发者拥有独立数据库环境, 互不干扰

决策 2: JavaScript Number 精度问题是否需要全局修复

- 决策内容: Jackson 全局将 Long 序列化为 String
- 为什么 AI 无法单独做这个决策: AI 需要被明确告知"Snowflake ID 超过 JS 安全整数范围"才会考虑修复。全局修改会影响所有 Long 字段的序列化行为
- 团队最终判断: 采用全局配置, 因为系统中所有 Long 类型字段 (userId、reqId、orderId 等) 都可能被前端接收, 统一序列化为 String 是最安全的做法

决策 3: main.js 与 main.ts 双入口保留还是合并

- 决策内容：暂时保留双入口，在 `main.js`（实际入口）中同步添加路由
- 为什么 AI 无法单独做这个决策：AI 不清楚两个文件的来源和历史原因，无法判断删除哪一个是否会影响其他成员的代码
- 团队最终判断：保留现状，在 `main.js` 中添加路由。待后续统一清理时由团队协商决定入口文件方案